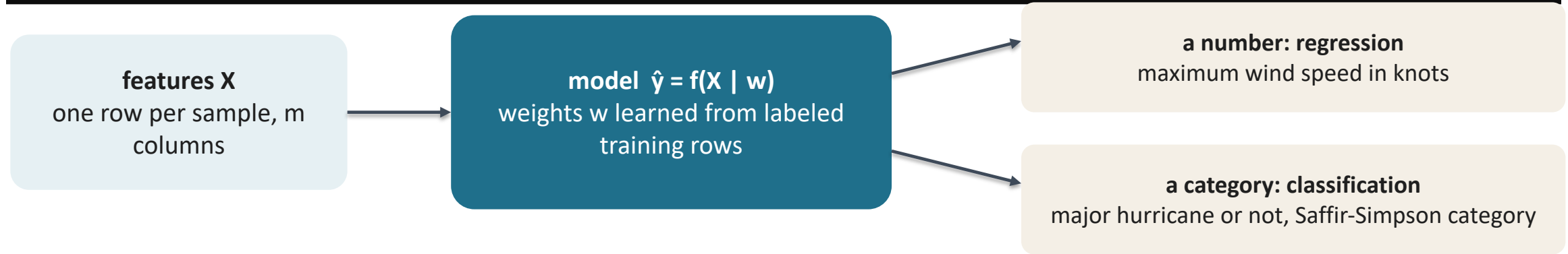


Supervised learning: regression and classification



The same recipe for every supervised model in this course

1. Choose a model family

straight line, logistic curve, tree, network

2. Choose a loss

how bad is one prediction:
squared error, cross-entropy

3. Minimize it on training rows

normal equation, or gradient descent

4. Judge on held-out rows

RMSE and R^2 , or precision, recall, AUC

This week's running example is the storm records from Weeks 2 and 3: regress maximum wind from pressure, location, and season, then classify major hurricanes and the full category scale.

Where these problems show up in our field

Regression: predict a number

Storm intensity

maximum sustained wind from central pressure and location

Crop yield

county maize yield from growing-season heat and rainfall

Hydrology

river discharge from rainfall and temperature

Climate emulation

global temperature maps from greenhouse gas concentrations

Classification: predict a category

Severity scales

Saffir-Simpson hurricane category, Enhanced Fujita tornado rating

Precipitation type

rain, snow, or freezing rain from the temperature profile

Did it happen?

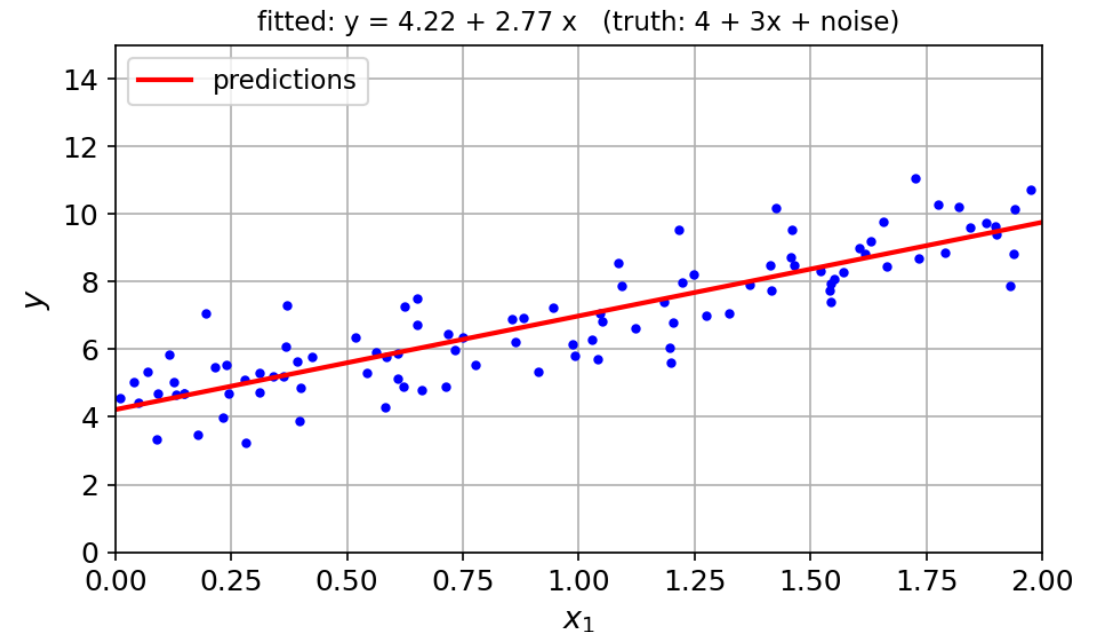
wildfire or no wildfire, flash flood or not: binary, and usually imbalanced

Labeling observations

land use, cloud type, or ice crystal shape from images

Linear regression: the model

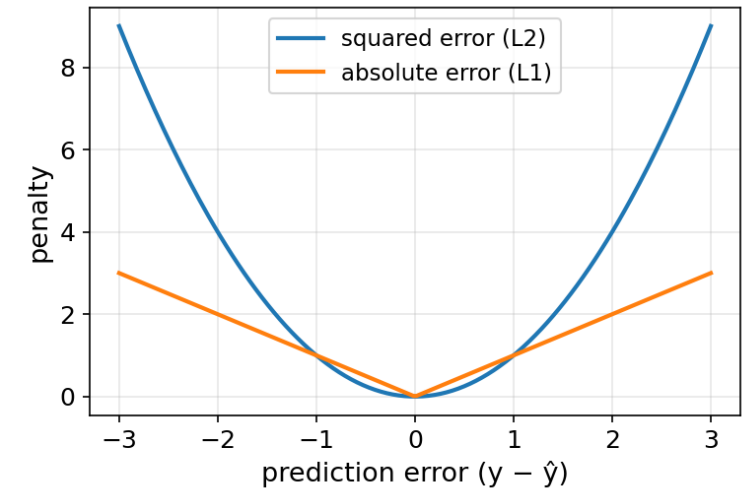
- Prediction is a weighted sum of the features plus a bias:
- $\hat{y} = b + w_1x_1 + w_2x_2 + \dots + w_mx_m = w \cdot x + b$
- **Each weight is the linear importance of one feature:** how much $\hat{y}$ changes per unit change in that feature, with the others held fixed
- The bias is the prediction when every feature is zero
- **In scikit-learn:** `LinearRegression().fit(X, y)`, then `coef_` and `intercept_`
- **For the storm records:** $\text{wind} = b + w_1 \cdot \text{pressure} + w_2 \cdot \text{latitude} + w_3 \cdot \text{longitude} + w_4 \cdot \text{season}$



Géron's toy data: 100 points from $y = 4 + 3x$ plus noise. The fit recovers roughly 4 and 3, not exactly, because the noise pulls the line.

Loss functions for regression

Loss	Formula	Units	Outliers
Mean squared error (L2)	$(1/N) \sum (y_i - \hat{y}_i)^2$	squared	punished hard: a 20 kt miss costs 400× a 1 kt miss
Mean absolute error (L1)	$(1/N) \sum y_i - \hat{y}_i $	original	punished linearly, so more robust
Root mean squared error	$\sqrt{\text{MSE}}$	original	same fit as MSE, readable number



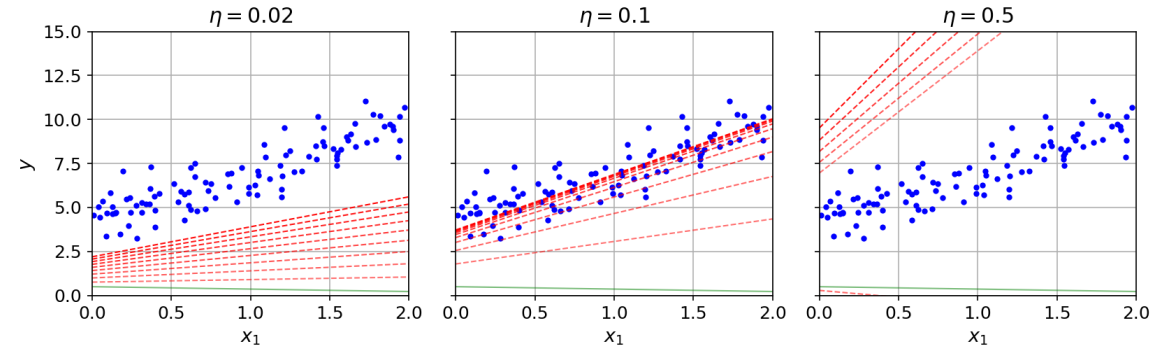
Fit with MSE, report RMSE. MSE has a clean minimum and a closed-form solution, and RMSE puts the answer back in knots.
Check MAE when a few wild samples dominate. If RMSE is much larger than MAE, a handful of storms are carrying most of the error.
Report RMSE and R^2 on the test set. Never on the training set: that number is about memorization, not prediction.

Fitting the weights

Normal equation, exact and closed form

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

- **Normal equation:** one line of algebra, no tuning. Costs an $m \times m$ inversion, so fine for a few features, hopeless for many. LinearRegression does this via a least-squares solver
- **Gradient descent:** start anywhere, step downhill on the loss, repeat. Scales to any size and is how every later model is trained
- **The learning rate η :** too small crawls, too large overshoots and diverges
- **Needs scaled features** (Weeks 2 and 3), or the loss surface is a narrow valley and the steps zigzag
- **Batch, stochastic, or mini-batch:** all rows, one row, or a few dozen rows per step

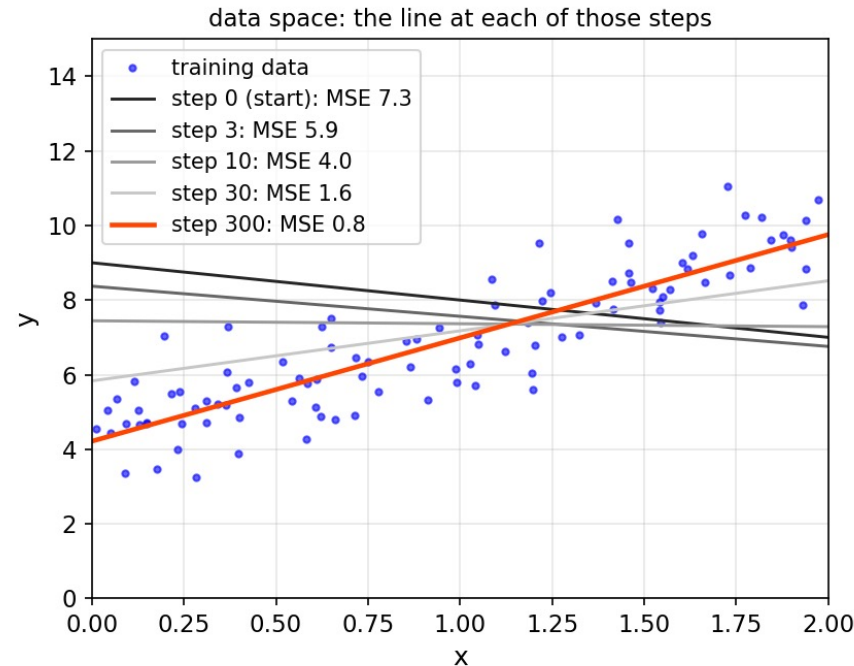
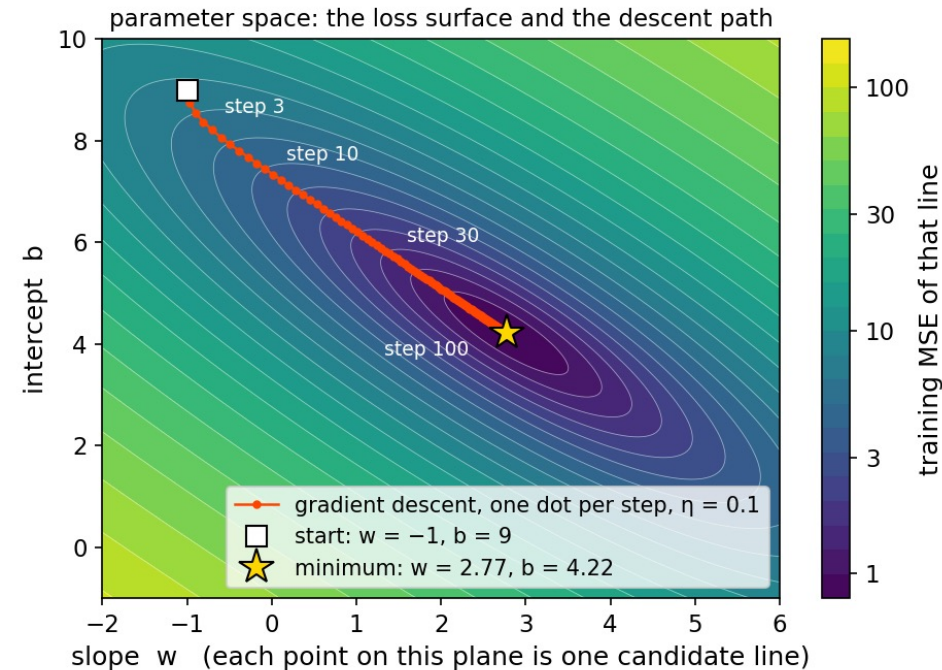


First ten steps of gradient descent from the same start. The green line is the starting guess, each dashed line one step. At $\eta = 0.02$ it crawls, at 0.1 it settles, at 0.5 it flies off the plot.

$$\nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} \text{MSE}(\boldsymbol{\theta}) \\ \frac{\partial}{\partial \theta_1} \text{MSE}(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial}{\partial \theta_n} \text{MSE}(\boldsymbol{\theta}) \end{pmatrix}$$

$$\boldsymbol{\theta}^{(\text{next step})} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta})$$

Gradient descent, step by step

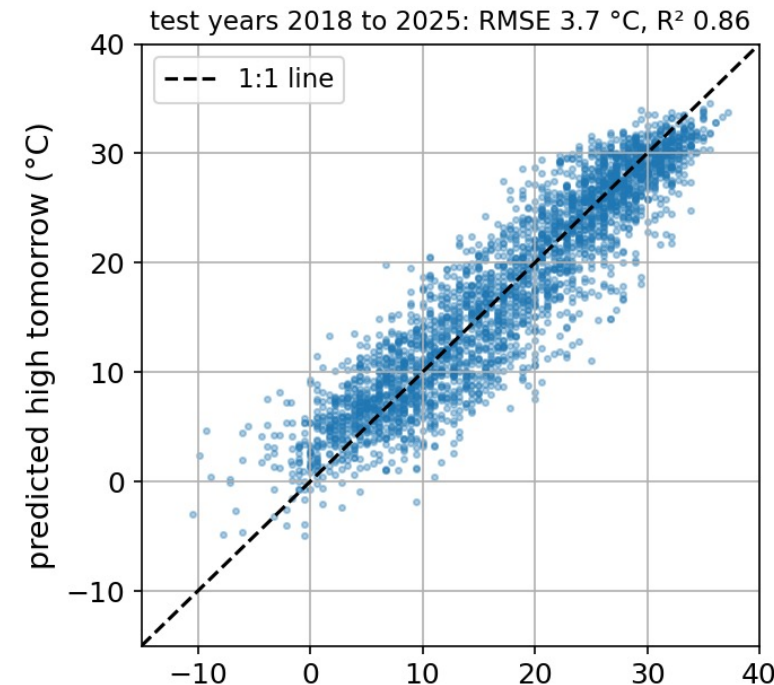


- **The axes are the parameters, not the data.** Every point on the left plane is one candidate line, slope w and intercept b . The color is that line's training MSE
- **A step goes against the gradient**, scaled by the learning rate. Long steps where the surface is steep, short ones as it flattens, so the points bunch up near the end
- **The right panel is the same steps seen on the data:** wrong-way line, flat line at the mean, then the fit
- **The bowl is a long diagonal valley**, which is why it takes about a hundred small steps. Standardizing x makes the bowl round and the descent fast

Step	Position (w, b)	Gradient ($\partial w, \partial b$)	Move
1	(-1.00, 9.00)	(-0.31, +2.48)	(+0.03, -0.25)
2	(-0.97, 8.75)	(-0.70, +2.04)	(+0.07, -0.20)
3	(-0.90, 8.55)	(-0.91, +1.77)	(+0.09, -0.18)

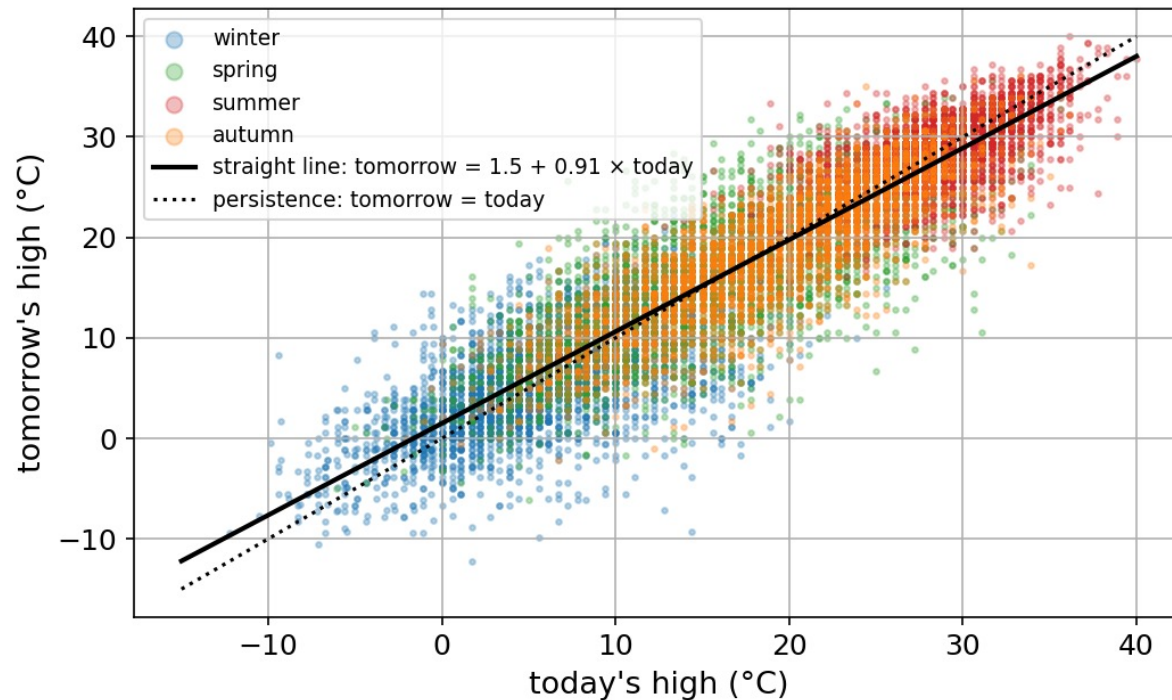
How good is it? RMSE, R^2 , and a baseline

- $R^2 = 1 - \Sigma(y_i - \hat{y}_i)^2 / \Sigma(y_i - \bar{y})^2$
- The fraction of the variance the model explains. 1 is perfect, 0 is no better than predicting the mean, negative is worse than the mean
- **Always compare with a baseline.** Predicting the mean high for every day is free. "Tomorrow is like today" is nearly as free and far better. A model that cannot beat those has shown nothing
- **Report on the test set.** Here the last eight years, never seen in training. Training R^2 measures memorization
- **Read the residuals, not just the number.** Spring days are the hardest to call (RMSE 4.5 °C), summer the easiest (2.6 °C): the atmosphere is steadier in July than in April



Model	Test RMSE	R^2
Always predict the mean high	9.7 °C	0.00
Climatology: mean for that day of year	4.6 °C	0.78
Persistence: tomorrow = today	4.3 °C	0.81
Linear regression, five features	3.7 °C	0.86

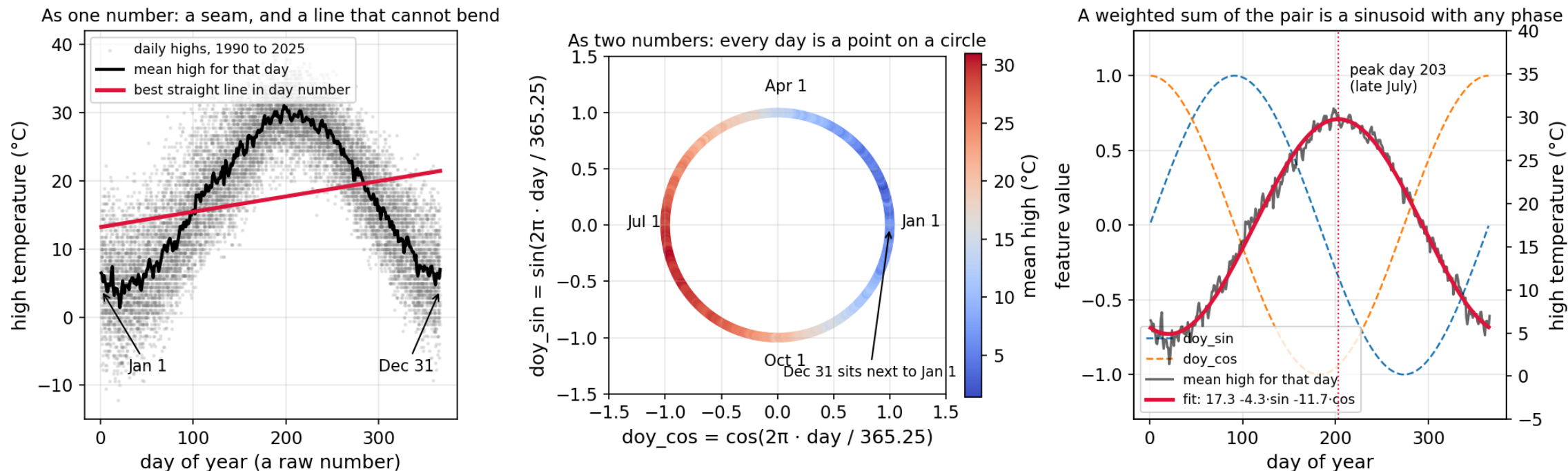
Reading the coefficients



The one-feature line has slope 0.91, not 1. Persistence is slightly too bold: the fit pulls hot and cold extremes back toward the seasonal mean, which is where the other four features earn their keep.

- **Today's high dominates:** $+0.42$ °C per °C, with today's low adding $+0.28$. Warm days are followed by warm days: the atmosphere has memory
- **The season terms encode regression to the norm:** the day-of-year cosine is worth -3.9 °C per unit. A 20 °C day in January is unusually warm and tends to cool tomorrow; the same day in July tends to warm
- **Units matter.** Rain looks negligible at -0.03 °C per mm. Standardized, today's high is worth 4.2 °C per standard deviation, the low 2.5 , the seasonal cosine 2.8 , rain 0.3
- **Association, not cause.** The season coefficients do not make tomorrow warmer. They describe where the record tends to return, given today

Cyclic features: the day of the year is a circle

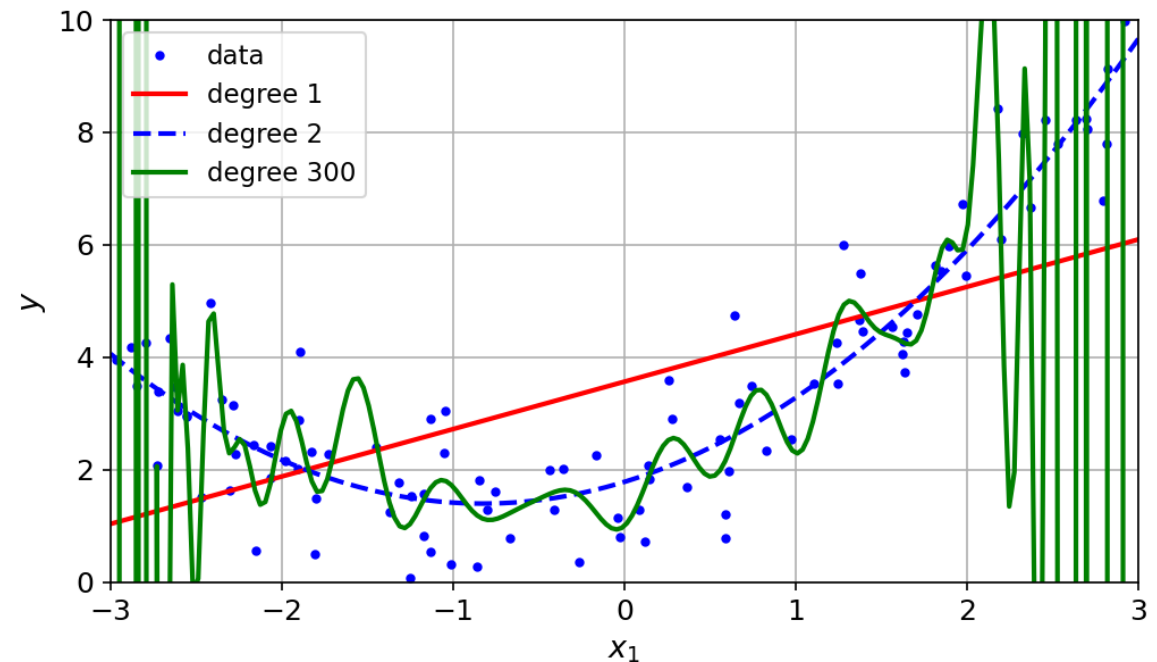


- **Day number has a seam:** December 31 and January 1 are 364 apart to the model and one day apart in reality, so a line through the year cannot bend and learns almost nothing
- **Two numbers fix it:** $\sin(2\pi \cdot \text{day} / 365.25)$ and $\cos(2\pi \cdot \text{day} / 365.25)$ place every day on a circle. Both are needed, or spring folds onto autumn
- **A weighted sum of the pair is a sinusoid with free height and timing,** so two coefficients capture the annual cycle and its month-long lag behind the sun. Same trick for hour of day, month, and wind direction

NOAA daily summaries for Central Park (station USW00094728), 1990 to 2025, the data set of the Week 4 live-coding tutorial; the right panel fits the mean high from the two season features alone

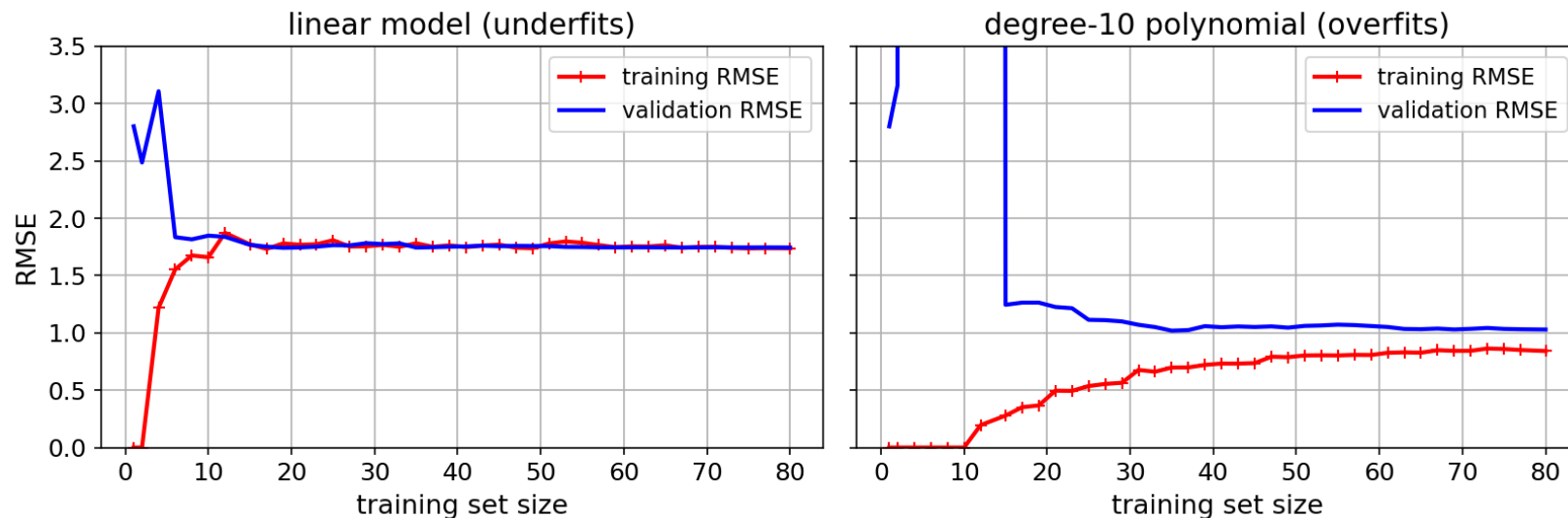
Polynomial regression: same machinery, curved fits

- **Add powers of the features**, x^2 , x^3 , and cross terms, then fit linear regression on the expanded matrix. `PolynomialFeatures` does the expansion
- **Still linear in the weights**, so the normal equation and gradient descent work unchanged
- **Degree is a complexity knob**: 1 cannot bend (underfits), 2 is right there, 300 threads the noise (overfits)
- **Training error always falls as degree rises**, so the degree must be chosen on validation data, never on the training set



Data from $y = 0.5x^2 + x + 2$ plus noise. The degree-2 fit recovers roughly those coefficients. The degree-300 fit has one weight per wiggle.

Underfitting and overfitting: read the learning curves

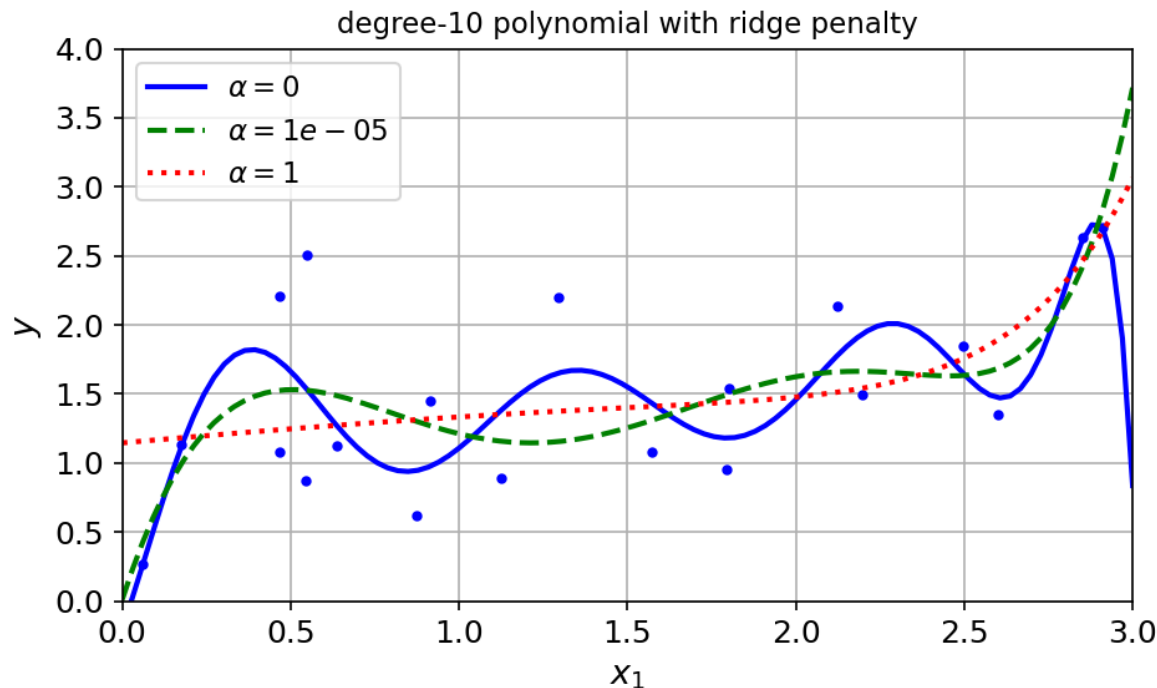


- **Left, underfitting:** both curves plateau high and close together. The model cannot fit even its training data. More rows will not help; a more flexible model or better features will
- **Right, overfitting:** training error far below validation error, with a gap that closes only slowly. More data helps, and so does regularization
- **The signature to memorize:** high training score, low test score means overfitting. Low on both means underfitting

Regularization: ridge and lasso

- **Add a penalty on the weights to the loss**, so a large coefficient must earn its keep
- **Ridge (L2 norm)**: penalty $\propto \sum w_j^2$. Shrinks every coefficient toward zero, keeps every feature
- **Lasso (L1 norm)**: penalty $\propto \sum |w_j|$. Drives coefficients to exactly zero: built-in feature selection
- **Needs standardized features**, because the penalty cannot tell knots per hPa from knots per yr

Ridge on a degree-10 polynomial, 20 points (Géron)



Equation 4-8. Ridge regression cost function

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \frac{\alpha}{m} \sum_{i=1}^n \theta_i^2$$

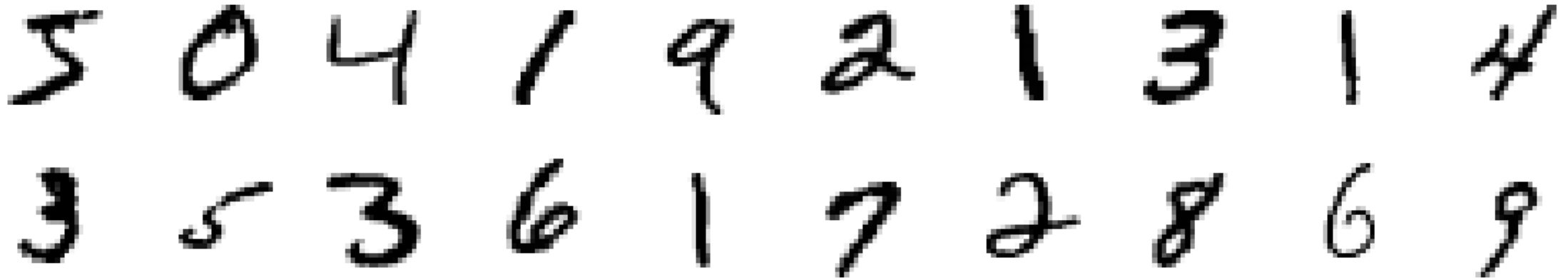
Equation 4-10. Lasso regression cost function

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + 2\alpha \sum_{i=1}^n |\theta_i|$$

Equation 4-12. Elastic net cost function

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + r(2\alpha \sum_{i=1}^n |\theta_i|) + (1-r)\left(\frac{\alpha}{m} \sum_{i=1}^n \theta_i^2\right)$$

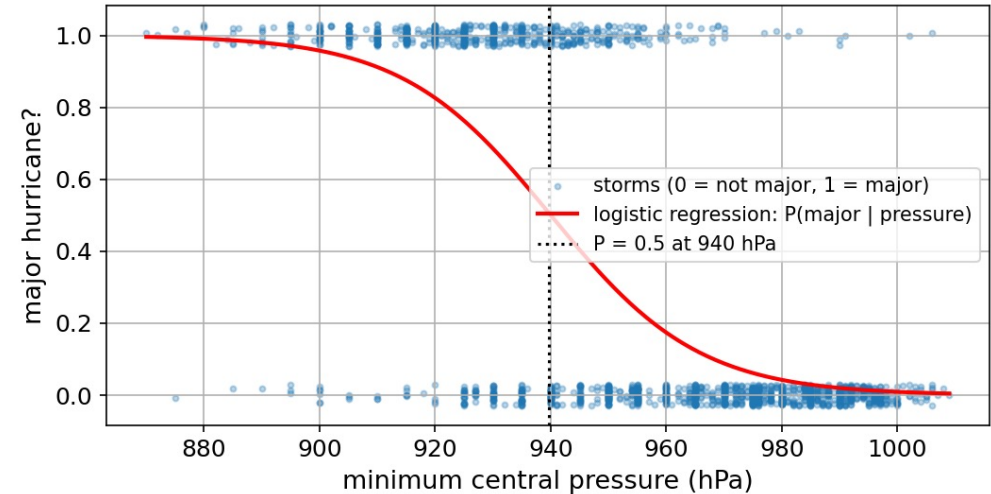
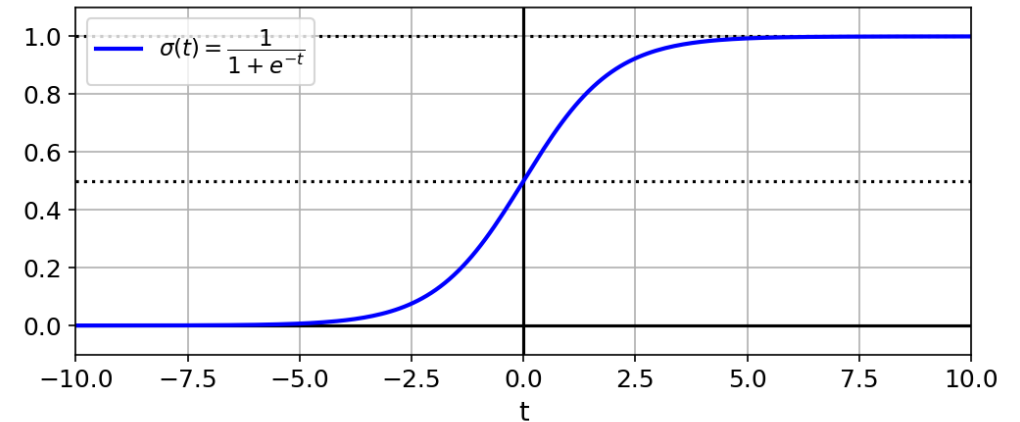
Classification: predicting a category



- **Binary:** one of two classes. Is this a 5? Is this storm a major hurricane (96 kt or more)?
- **Multi-class:** one of several exclusive classes. Which digit? Which Saffir-Simpson category, 0 to 5?
- **Multi-label:** several non-exclusive tags per sample, an image with both cloud and snow. Not this week
- **The classic test bed:** MNIST, 70,000 handwritten digits, 784 pixel features, ten classes. The next four slides use it

Logistic regression: score, then squash

- **Score, then squash:** $z = w \cdot x + b$, then $p = \sigma(z) = 1 / (1 + e^{-z})$
- **p is the probability of the positive class.** Predict positive when $p \geq 0.5$, which is $z \geq 0$. The boundary $z = 0$ is a straight line, a plane in more dimensions
- **Why scaling matters here:** scikit-learn regularizes logistic regression by default, and fits it with an iterative solver. Both want features on one scale
- **make_pipeline(StandardScaler(), LogisticRegression())** fits the scaler on the training rows only



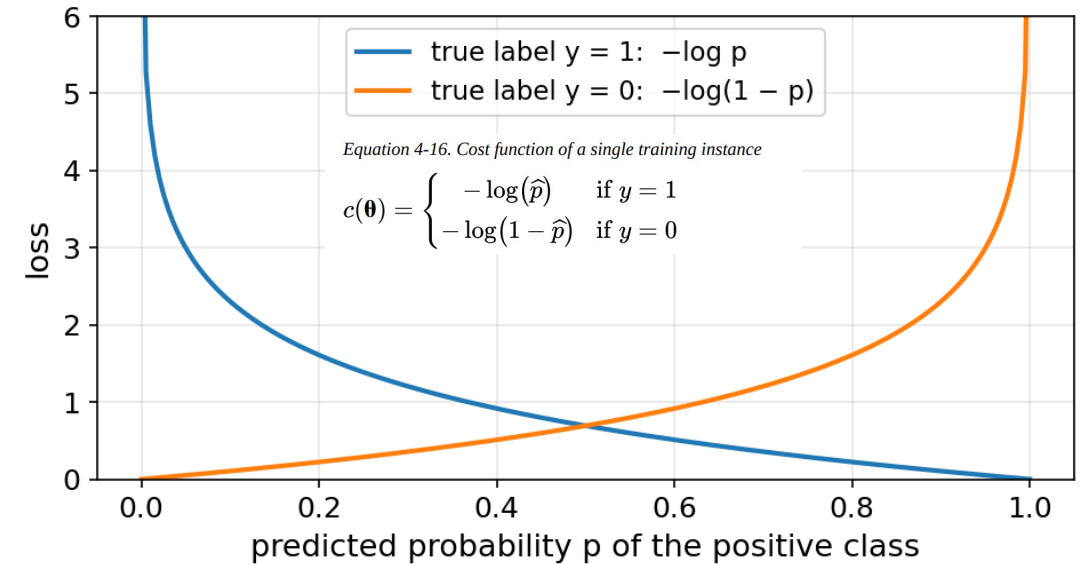
Top: Figures regenerated from the author's notebook for Géron, Hands-On Machine Learning, 3rd ed., Ch. 4, Training Models (Apache 2.0 code). Bottom: IBTrACS v04r01 named storms, one row per storm (NOAA NCEI)

Training a classifier: the cross-entropy loss

- **Counting mistakes has no slope**, so we minimize binary cross-entropy instead:
- $L = -[y \log p + (1 - y) \log(1 - p)]$
- **Confident and right costs almost nothing.** Confident and wrong costs without limit. That is what keeps the probabilities calibrated
- **No closed form**, so gradient descent: batch, stochastic, or mini-batch. The loss is convex, one bowl, so it finds the global minimum

Equation 4-17. Logistic regression cost function (log loss)

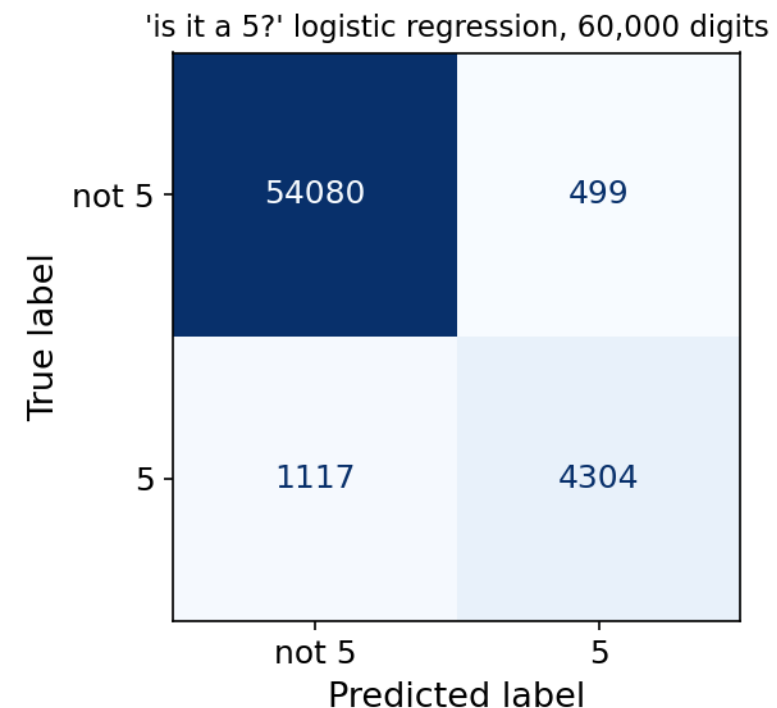
$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m [y^{(i)} \log(\hat{p}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)})]$$



Read the curve at $p = 0.01$ for a true positive: the loss is about 4.6, against 0.7 for a prediction of 0.5. Wrong and sure is the expensive mistake.

Accuracy is not enough: the confusion matrix

- **Ten percent of digits are 5s**, so "never a 5" scores 90% accuracy and has learned nothing. Accuracy hides imbalance
- **Rows are truth, columns are predictions.** Off the diagonal are two different mistakes: false alarms (FP) and misses (FN)
- **Precision = $TP / (TP + FP)$:** of the positives I called, how many were right
- **Recall = $TP / (TP + FN)$:** of the real positives, how many did I catch
- **F1 = harmonic mean of the two**, when both kinds of mistake matter
- **Storms:** 36% are major hurricanes, so "never major" scores 64%. Beat that before claiming anything



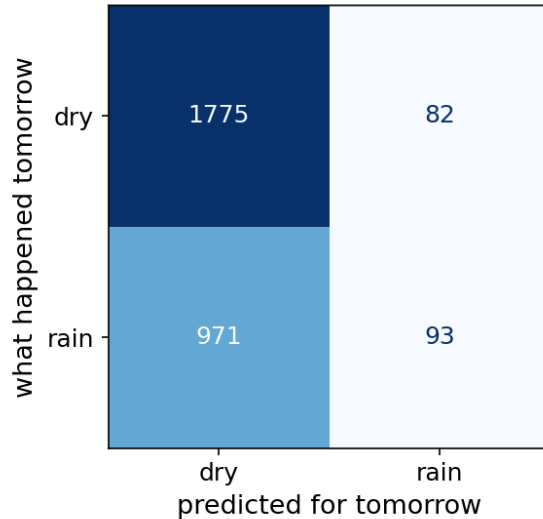
5-detector, cross-validated	Precision	Recall	Accuracy
Logistic regression	0.90	0.79	0.97
"never a 5"	0	0	0.91

Figures regenerated from the author's notebook for Géron, *Hands-On Machine Learning*, 3rd ed., Ch. 3, Classification (Apache 2.0 code; MNIST digits)

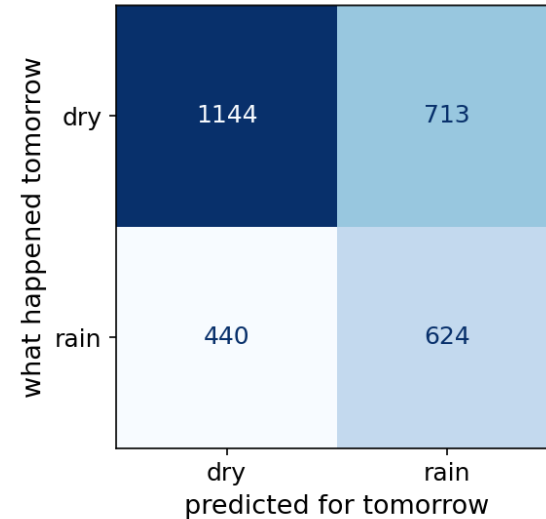
The same matrix on our data: will it rain tomorrow?

Will it rain tomorrow at Central Park? Logistic regression on 2018 to 2025 test days

threshold 0.5: accuracy 0.64, precision 0.53, recall 0.09



threshold 0.35: accuracy 0.61, precision 0.47, recall 0.59

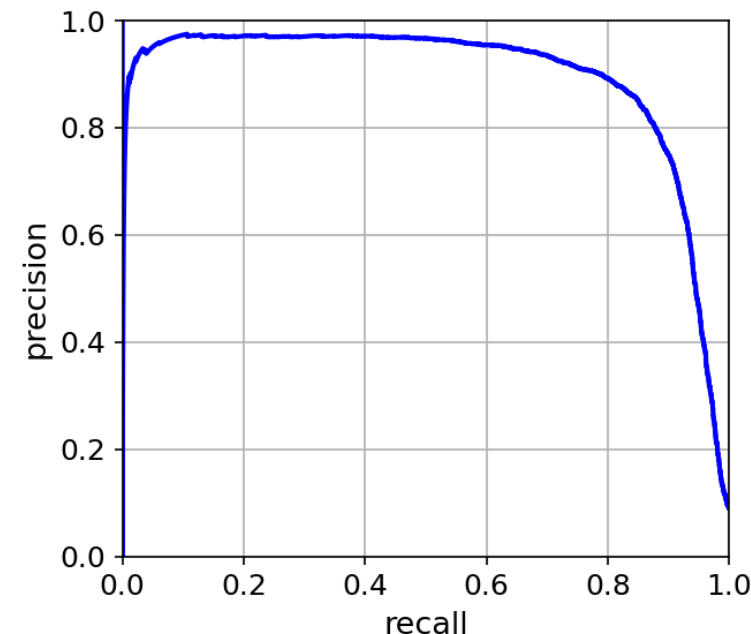
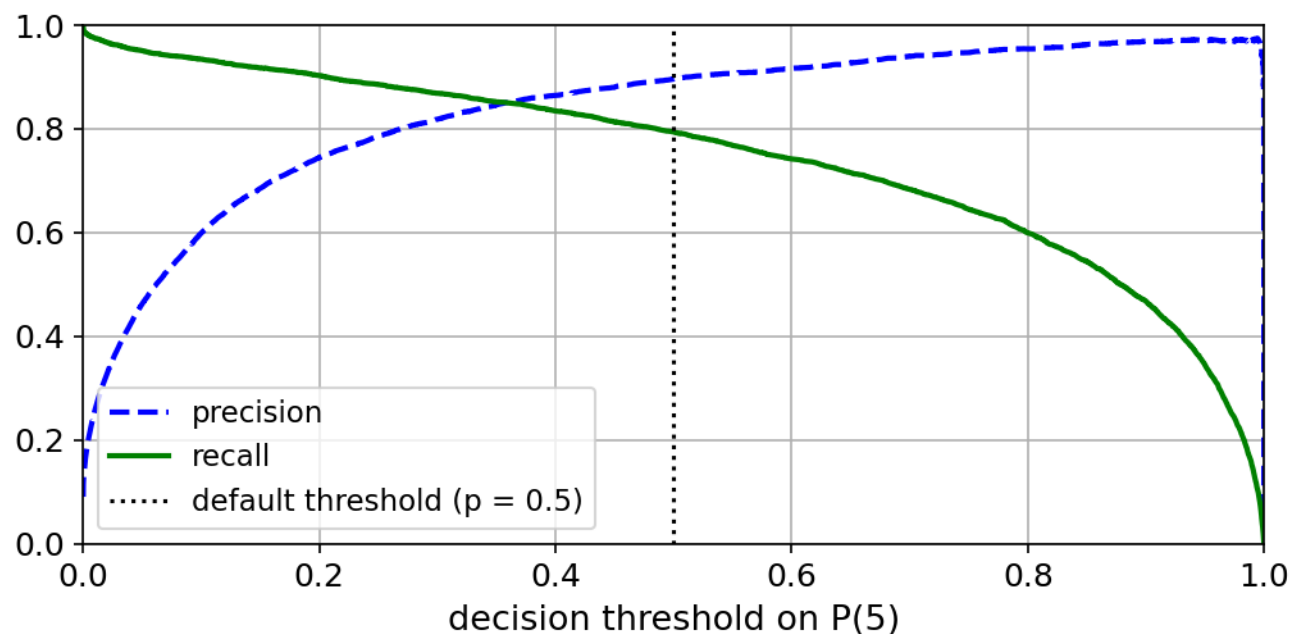


- **Rain is the minority:** 36% of test days, so "always dry" scores 0.64. At the default cutoff the model scores 0.64 too, catching 93 of 1,064 rainy days
- **Same model, cutoff at 0.35:** catches 624 rainy days at the price of 713 false alarms. Accuracy falls, the forecast becomes useful
- **The probabilities reflect the difficulty.** Yesterday's weather barely predicts tomorrow's rain, so most of them sit between 0.2 and 0.4
- **Which matrix is better?** Only a cost can say: an umbrella wants recall, a wedding wants precision

Cutoff	Called rain	Caught	Missed	False alarms	Accuracy
0.50	175	93	971	82	0.64
0.35	1,337	624	440	713	0.61

NOAA daily summaries for Central Park (station USW00094728); logistic regression on today's high, low, rainfall, and season, trained on 1990 to 2017, tested on 2018 to 2025, the Week 4 live-coding tutorial

Precision or recall? A domain choice

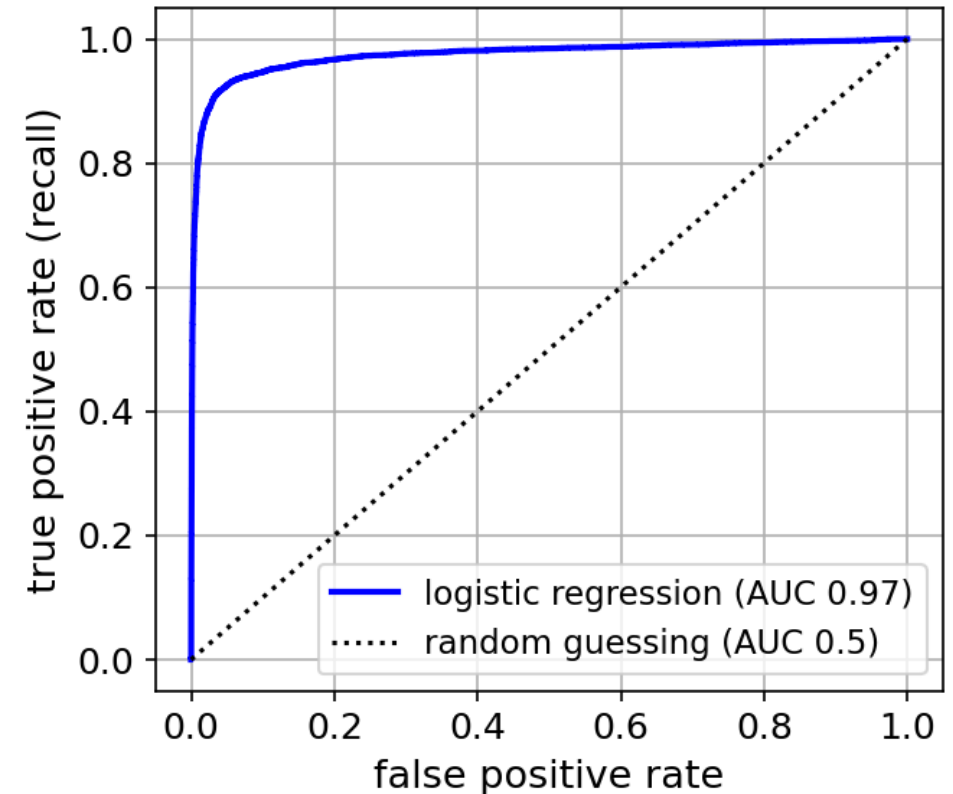


- **The threshold is yours to set.** Raise it: fewer false alarms, more misses. Lower it: the reverse. You choose a point on the right-hand curve
- **Favor recall (fewer misses) when a miss is costly:** evacuations from flash floods, dangerous storms, wildfires. Accept some false alarms to catch nearly every event
- **Favor precision when a false alarm is costly:** permafrost detection, siting a well. If the model says yes, it had better be there
- **An evacuation order sits far below $p = 0.5$,** which only works if the probability is calibrated

Figures regenerated from the author's notebook for Géron, Hands-On Machine Learning, 3rd ed., Ch. 3, Classification (Apache 2.0 code; MNIST digits)

The ROC (Receiver Operating Characteristic) curve and AUC

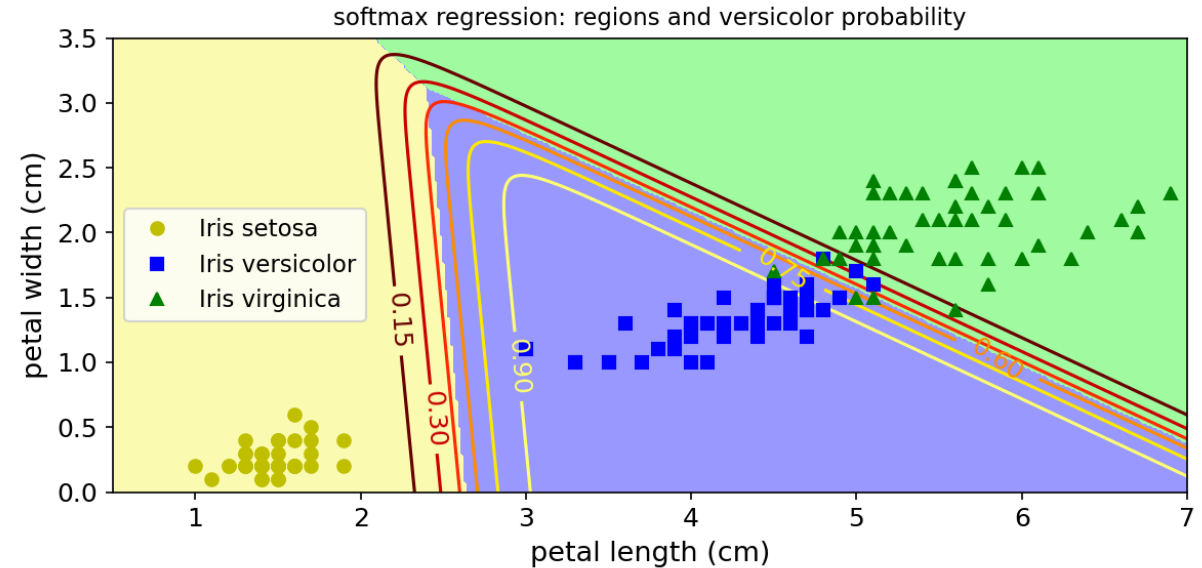
- **Sweep the threshold**, plot true positive rate (recall) against false positive rate (share of real negatives wrongly flagged)
- **Perfect hugs the top-left corner.** Random guessing is the diagonal. Real models sit between
- **AUC, the area under it**, is one threshold-free number: 0.5 is a coin flip, 1 is perfect. The 5-detector scores 0.97; pressure alone gives 0.91 for major hurricanes
- **It cannot be gamed by predicting the majority class**, so report it next to accuracy
- **Caveat:** with rare positives the ROC curve flatters. Prefer the precision-recall curve there



Figures regenerated from the author's notebook for Géron, Hands-On Machine Learning, 3rd ed., Ch. 3, Classification (Apache 2.0 code; MNIST digits)

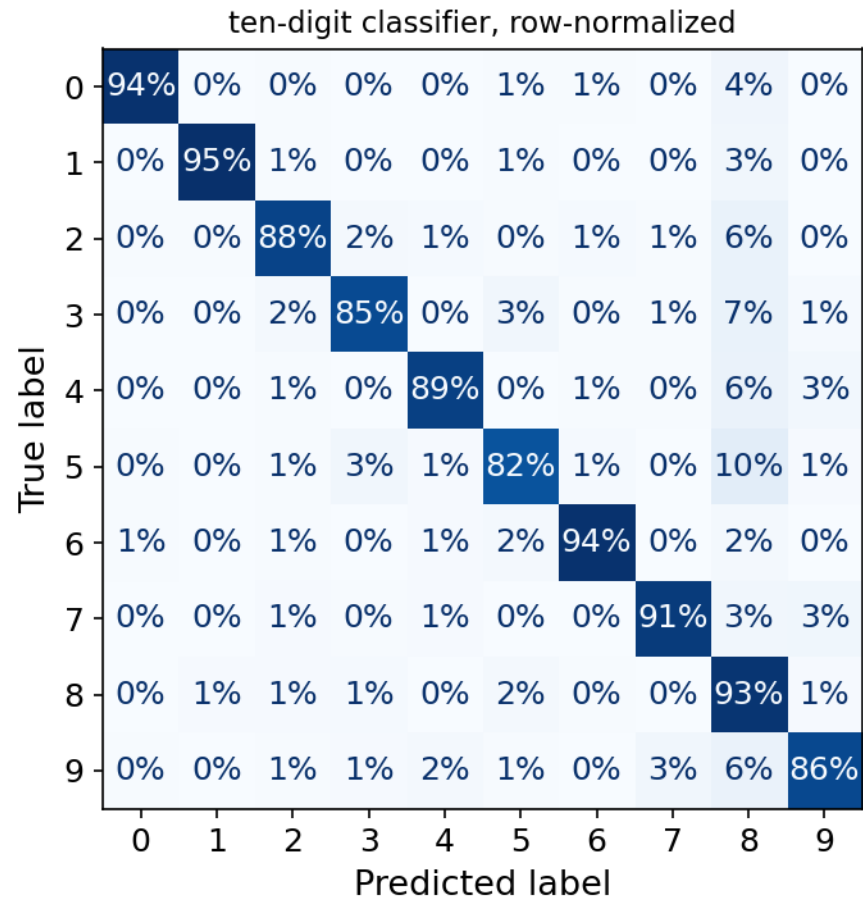
Softmax regression: many classes at once

- **One linear score per class**, $z_k = w_k \cdot x + b_k$, then normalize:
- $p_k = \exp(z_k) / \sum_j \exp(z_j)$
- **Probabilities are positive and sum to one.** Predict the class with the largest. Train with cross-entropy, $-\log$ of the true class's probability
- **Boundaries are still straight**, because each pairwise comparison is linear
- **LogisticRegression does this automatically** when y has more than two classes, such as the six Saffir-Simpson categories



Three iris species from two petal measurements. Background is the predicted class, contours the probability of versicolor. Every boundary is a straight line.

Reading a multiclass confusion matrix



- **Normalize by row** so each cell is the share of a true class going to each prediction, otherwise big classes dominate
- **The errors are the information.** Here 5s and 3s are read as 8s: an 8 looks like most digits plus a little ink
- **For storm categories:** confusions will be between adjacent categories. Category 2 is only 13 kt wide and the regression error is 14 kt
- **Classify directly, or regress wind then threshold?** Direct wins on exact category (0.58 vs 0.53); both are within one category 93% of the time

Figures regenerated from the author's notebook for Géron, Hands-On Machine Learning, 3rd ed., Ch. 3, Classification (Apache 2.0 code; MNIST digits); storm numbers from the same IBTrACS records

Explain or predict?

Explanatory model

Pressure and wind are measured on the same storm at the same time. The model has learned the wind-pressure relationship: real physics, and how intensity is estimated when only one can be measured.

Predictive model

A forecast for 24 hours ahead cannot use a pressure reading from 24 hours ahead. Features must be known at forecast time: current state, sea surface temperature, wind shear. And the split must respect time (Week 6).

Is the record homogeneous? IBTrACS merges agencies whose methods changed over a century: ship reports, then aircraft, then satellite estimates. One-minute sustained winds in the US, ten-minute elsewhere, a 10% offset on its own.

So the season coefficient is partly a proxy for observing technology, and any trend in intensity is confounded with our improving ability to see it. Check by fitting separately by era or basin and watching the coefficients.

The metrics say how well the function fits. They cannot say whether it answers the right question.